

Lab 4: Pets Expression Classification

Nour Mohamed Mahmoud 7591

Ziad Mohamed Fahmy 7345

Mahmoud Mohamed Elzamly 7871

Introduction

This report details the process of building and evaluating a deep learning model for Pets Expression Classification. The task involves classifying images into predefined categories using a VGG-like neural network architecture. The report outlines data preparation, augmentation techniques, model design, hyperparameters, and the obtained results.

Data Preparation

Dataset Description

The dataset consists of images grouped into 4 distinct classes, each representing a specific category. The data is split into three subsets:

- **Training Set:** Used for model training.
- **Validation Set:** Used for tuning model hyperparameters and preventing overfitting.
- **Test Set:** Used for final evaluation of the model.

Data Preprocessing Steps

Normalization:

- Training set pixel values were normalized using ImageNet mean and standard deviation ($[0.485, 0.456, 0.406]$ and $[0.229, 0.224, 0.225]$, respectively).

- Validation and test sets were normalized to have a mean of 0.5 and a standard deviation of 0.5 for all channels.

Resizing:

- All training images were resized to 224x224 to ensure uniformity and compatibility with the VGG16 model input dimensions.

Data Splitting:

- The dataset was split into training, validation, and test sets, ensuring class balance.

Data Augmentation

To improve the model's robustness and reduce overfitting, data augmentation techniques were applied to the training images. These included:

- **Random Horizontal Flip:** Flipping images horizontally with a probability of 50%.
- **Random Rotation:** Rotating images randomly within a range of ± 10 degrees.
- **Color Jitter:** Randomly adjusting brightness, contrast, and saturation.
- **Random Cropping:** Randomly cropping regions of the image and resizing back to 224x224.

Data augmentation was implemented using the `torchvision.transforms` library to generate diverse training samples.

Model Architecture

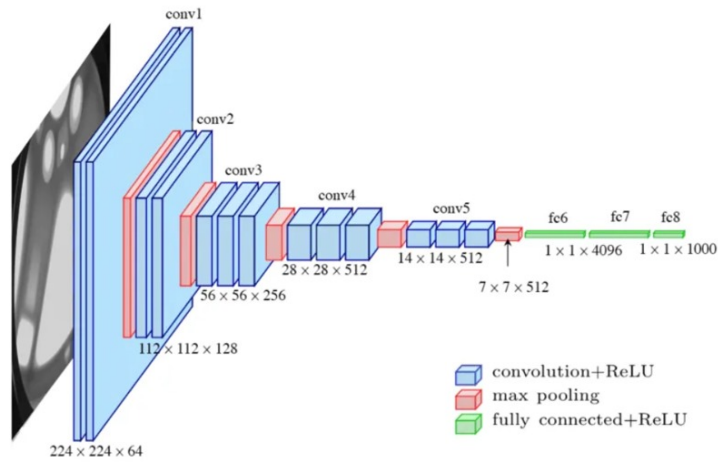
The VGG16 neural network model was implemented for this task. The key components of the architecture include:

- **Convolutional Layers:** Three convolutional blocks, each consisting of two convolutional layers with ReLU activation and batch normalization. Each block ends with a max-pooling layer for spatial downsampling.

- **Fully Connected Layers:** The output of the convolutional layers was flattened and passed through two fully connected layers with ReLU activation. Dropout was used to reduce overfitting.
- **Output Layer:** A final fully connected layer with a softmax activation function to predict probabilities for each class.

Model Summary:

- Input Size: $224 \times 224 \times 3$
- Number of Parameters: Approximately 20 million



Hyperparameters

The following hyperparameters were used:

- Learning Rate: 0.0001 (adjusted dynamically using a learning rate scheduler).
- Batch Size: 32
- Number of Epochs: 20
- Optimizer: Adam optimizer with default β values ($\beta_1 = 0.9$, $\beta_2 = 0.999$).
- Loss Function: Cross-entropy loss

Training Process

Training Loop

- **Forward Pass:** The input images were passed through the network to compute predicted probabilities.
- **Loss Calculation:** Cross-entropy loss was computed between predicted and true labels.
- **Backward Pass:** Gradients were computed via backpropagation.
- **Optimization:** The weights were updated using the Adam optimizer.

Validation Loop

After each epoch, the model was evaluated on the validation set to compute:

- Validation loss
- Validation accuracy

Training and validation losses were monitored to detect overfitting or underfitting.

Results

Test Accuracy

The model achieved a test accuracy of 34.21%.

Evaluation Metrics

A detailed evaluation of the model was performed using precision, recall, F1-score, and the confusion matrix.

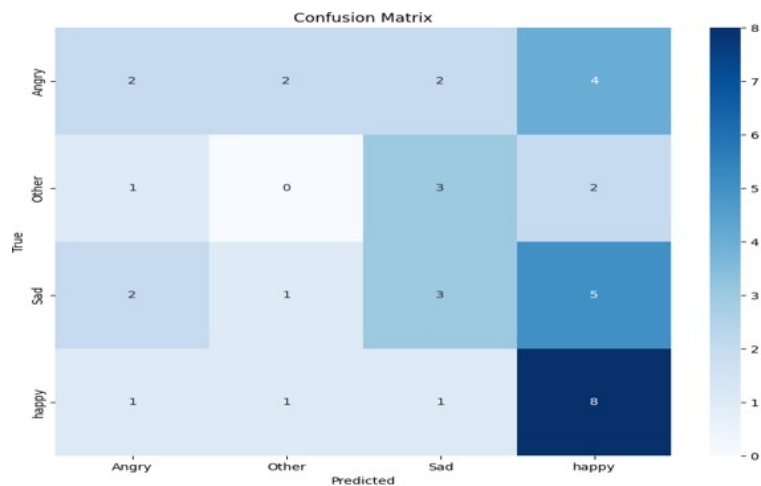
Classification Report

Classification report:

Confusion Matrix

A confusion matrix was plotted to visualize the model's predictions and identify any class imbalances or frequent misclassifications.

Classification Report:				
	precision	recall	f1-score	support
Angry	0.3333	0.2000	0.2500	10
Other	0.0000	0.0000	0.0000	6
Sad	0.3333	0.2727	0.3000	11
happy	0.4211	0.7273	0.5333	11
accuracy			0.3421	38
macro avg	0.2719	0.3000	0.2708	38
weighted avg	0.3061	0.3421	0.3070	38



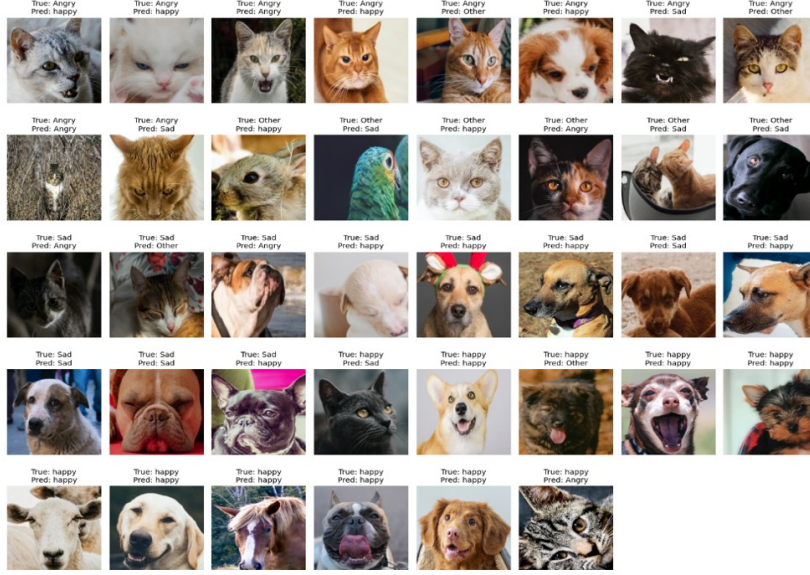
Visualizations

To assess model predictions, test images were displayed alongside their true and predicted labels. This qualitative analysis highlighted the model's strengths and potential areas for improvement.

MobileNet: A Lightweight Neural Network for Mobile and Embedded Devices

Introduction

MobileNet is a class of efficient neural network architectures designed specifically for mobile and embedded vision applications. Developed by researchers at Google, MobileNet addresses the challenge of deploying deep learning models on devices with limited computational power and memory. It achieves this



by utilizing depthwise separable convolutions, a technique that significantly reduces the number of parameters and computational complexity compared to traditional convolutional neural networks (CNNs).

Architecture

The key innovation in MobileNet is its use of depthwise separable convolutions, which decompose a standard convolution operation into two simpler operations:

- **Depthwise Convolution:** Applies a single filter to each input channel independently.
- **Pointwise Convolution:** Uses a 1x1 convolution to combine the outputs of the depthwise convolution into a new feature map.

By separating these two operations, MobileNet reduces the computational cost and the number of parameters while maintaining good accuracy.

Depthwise Separable Convolution

Given an input tensor with dimensions $H \times W \times C$ and a filter size of $K \times K \times C$, a standard convolution would require:

$K^2 \times C^2$ operations.

However, depthwise separable convolutions require:

$K^2 \times C$ operations, resulting in significant computational savings.

Width and Resolution Multipliers

MobileNet introduces two hyperparameters to further tailor the model to specific use cases:

- **Width Multiplier (α):** Reduces the number of channels in each layer.
- **Resolution Multiplier (ρ):** Reduces the input image resolution.

These multipliers allow developers to trade off between model size, computational cost, and accuracy.

Applications

MobileNet is widely used in applications where computational resources are constrained, such as:

- Real-time object detection and classification on smartphones.
- Embedded vision systems for robotics and IoT devices.
- Augmented reality and virtual reality applications.

Strengths and Limitations

Strengths:

- Lightweight and efficient, making it suitable for resource-constrained environments.
- Customizable with width and resolution multipliers.
- Pretrained models available for transfer learning.

Limitations:

- Slightly lower accuracy compared to larger networks like ResNet and VGG.
- Performance depends on the choice of multipliers, requiring careful tuning.

ResNet: A Deep Residual Learning Framework

Introduction

ResNet (Residual Network) revolutionized deep learning by addressing the problem of vanishing gradients in very deep neural networks. Proposed by Kaiming He et al. in 2015, ResNet introduced the concept of residual connections, enabling the training of networks with hundreds or even thousands of layers. This innovation led to state-of-the-art performance across various tasks, including image classification, object detection, and semantic segmentation.

Architecture

The primary building block of ResNet is the residual block, which incorporates skip connections to allow gradients to flow directly through the network during backpropagation.

Residual Block

A standard residual block consists of:

- Two or three convolutional layers with batch normalization and ReLU activation.
- A skip connection that bypasses these layers, adding the input directly to the output.

Mathematically, the output of a residual block can be expressed as:

$$\text{Output} = F(\text{Input}) + \text{Input},$$

where F represents the output of the convolutional layers, and Input is the input. This identity mapping enables the network to learn residual functions, which are easier to optimize than direct mappings.

Network Variants

ResNet has several variants, differing in the number of layers:

- **ResNet-18 and ResNet-34:** Shallow versions suitable for less complex tasks.

- **ResNet-50, ResNet-101, and ResNet-152:** Deeper versions for complex tasks, featuring bottleneck layers to reduce computational cost.

Bottleneck Layers

In deeper versions like ResNet-50, a bottleneck block is used, consisting of three convolutions:

- 1x1 convolution to reduce dimensionality.
- 3x3 convolution for feature extraction.
- 1x1 convolution to restore dimensionality.

This structure reduces the computational burden while maintaining accuracy.

Applications

ResNet is widely used across various domains, including:

- Image recognition and classification.
- Object detection frameworks like Faster R-CNN and Mask R-CNN.
- Medical imaging and autonomous driving systems.

Strengths and Limitations

Strengths:

- Ability to train very deep networks without vanishing gradients.
- High accuracy on complex tasks.
- Versatile and adaptable for various architectures and applications.

Limitations:

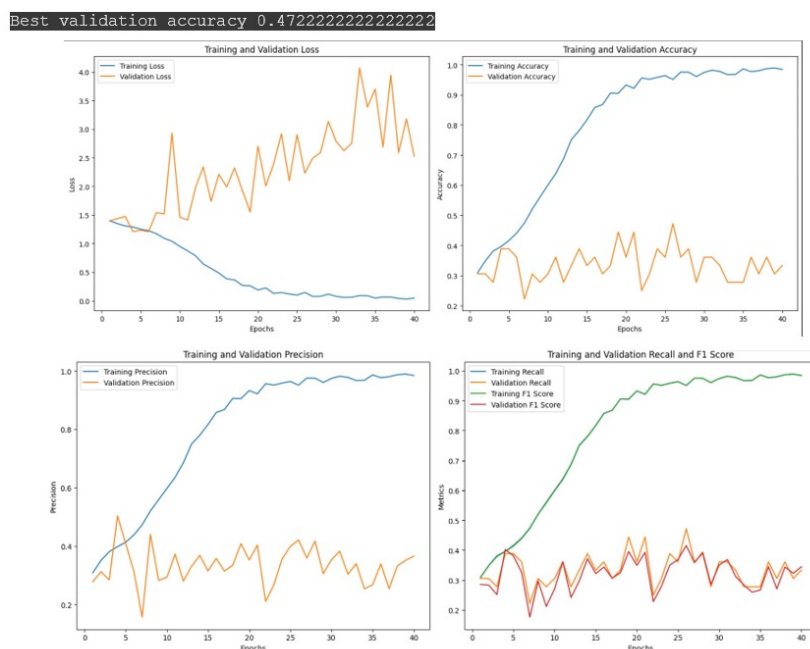
- Computationally expensive for deep variants like ResNet-152.
- Increased depth may lead to diminishing returns in accuracy improvement.

Conclusion

Both MobileNet and ResNet have their unique strengths and serve different purposes. While MobileNet excels in lightweight applications for resource-constrained environments, ResNet remains a powerhouse for high-performance tasks requiring deep architectures. The choice between the two depends on the specific requirements of the task, such as computational resources, latency, and accuracy.

Results for DenseNet 121:

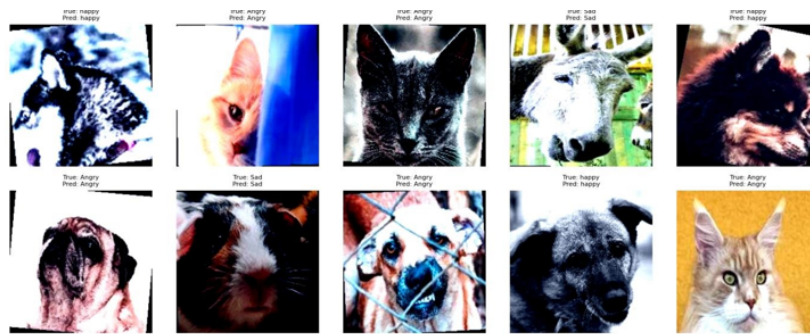
Implemented model training:



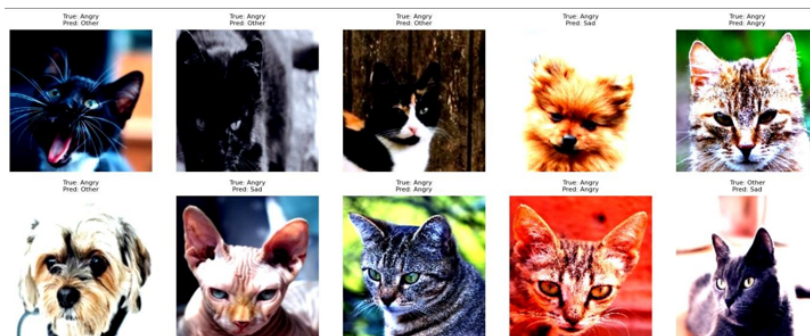
Performance on training images:

Pretrained model training:

Best validation accuracy 0.6111111111111112



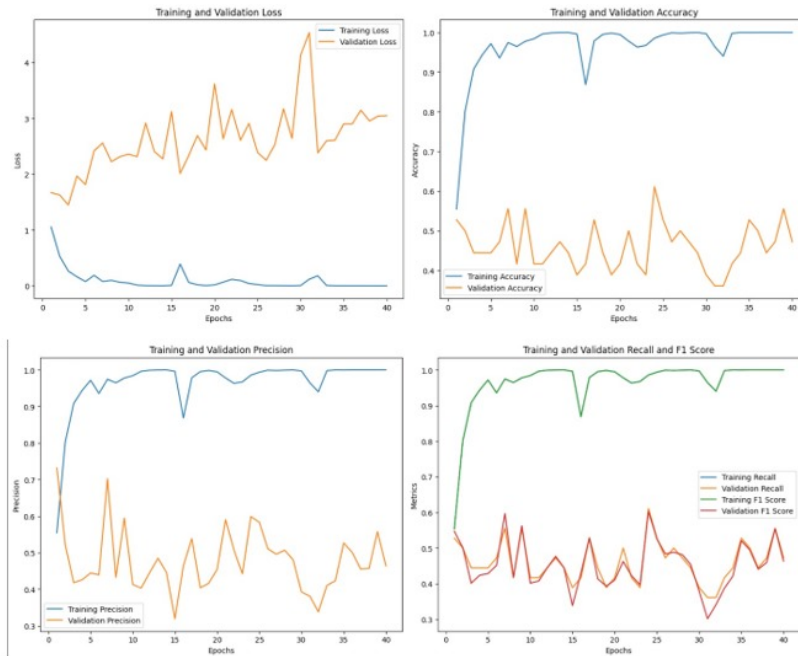
Performance on validation images:



Implemented and pretrained models' performance on test data:

Results for Inception v3:

Implemented model training: Best validation accuracy 0.5

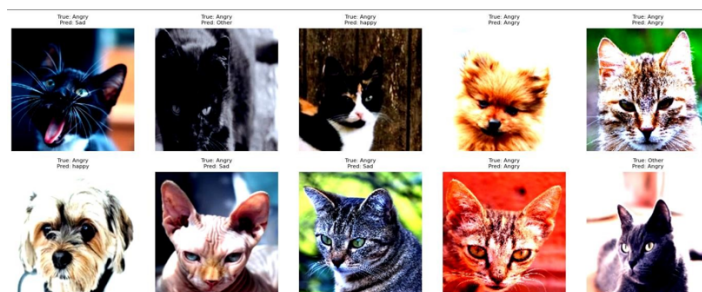
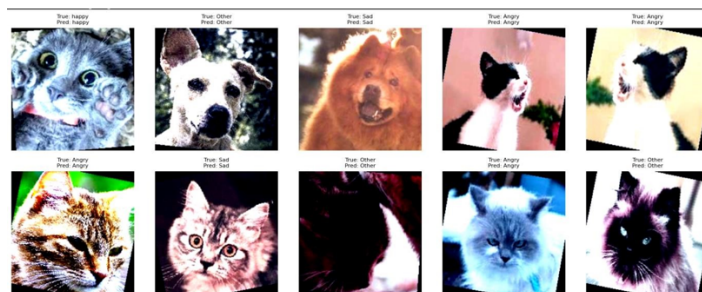
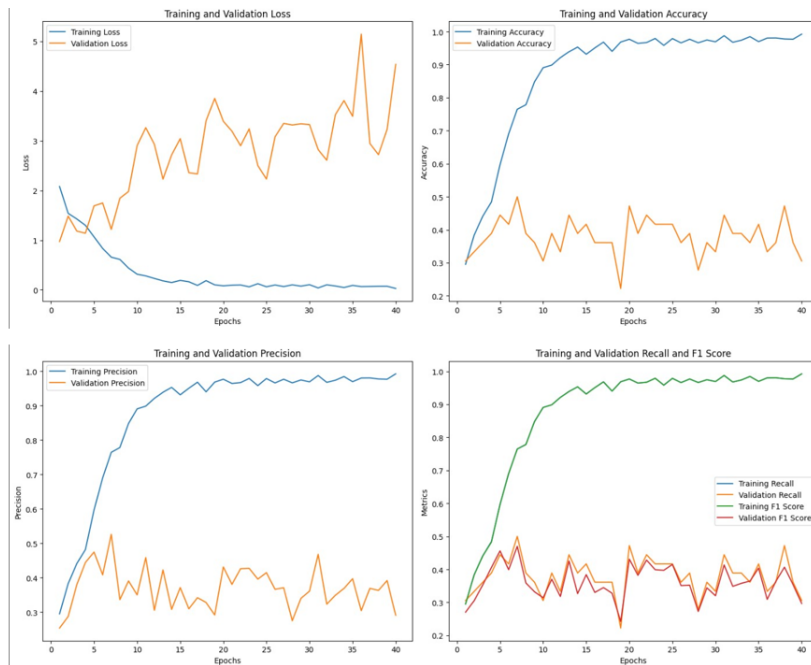


Implemented model performance:

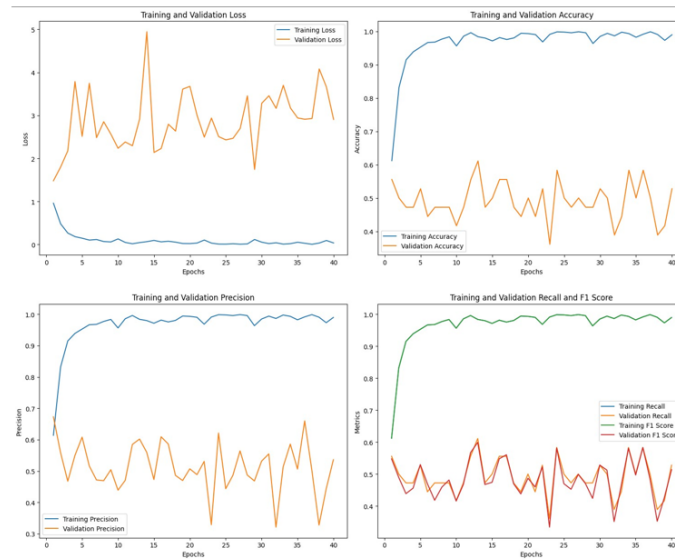
```
Test Loss: 5.3772
Test Accuracy: 0.1316
Test Precision: 0.1224
Test Recall: 0.1316 Test
F1 Score: 0.1268
Confusion Matrix:
[[2 1 2 5]
 [7 0 3 1]
 [8 3 1 4]
 [9 0 5 2]]
```

Pretrained model performance:

```
Test Loss: 2.2896
Test Accuracy: 0.5263
Test Precision: 0.5281
Test Recall: 0.5263 Test
F1 Score: 0.5216
Confusion Matrix:
[[4 1 2 3]
 [10 3 2 0]
 [11 2 5 2]
 [0 0 3 8]]
```



Best validation accuracy 0.6111111111111112



Implemented model performance:

Test Loss: 3.0588
 Test Accuracy: 0.3947
 Test Precision: 0.4229
 Test Recall: 0.3947
 F1 Score: 0.3798

Confusion Matrix:

```
[[4 2 2]
 [1 3 0]
 [1 2 6]]
```

Pretrained model performance:

Test Loss: 3.3126
 Test Accuracy: 0.3421
 Test Precision: 0.3803
 Test Recall: 0.3421
 F1 Score: 0.3439

Confusion Matrix:

```
[[4 0 2]
 [0 0 0]
 [0 3 3]]
```